



Lecture 2 : Voir sans apprendre

Vision embarquée à règles, projet EcoTri 4.0

Peut-on distinguer une canette d'un carton avec de simples seuils ?

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **L'image en mémoire** : *Que coûte une image, et quel format choisir sur un MCU ?*
- **Segmenter** : *Comment séparer l'objet du tapis avec des seuils, puis le décrire par quelques nombres ?*
- **Formes et codes** : *Comment trouver des contours, des cercles, des droites et lire un QR code ?*
- **Lumière et calibration** : *Pourquoi un seuil réglé le matin ne marche plus l'après-midi ?*
- **Limites et synthèse** : *Quel taux d'erreur atteint un classifieur à règles, et pourquoi faudra-t-il apprendre ?*

Objectifs

Ce que vous saurez faire en sortant de la salle

À la fin de cette lecture, vous serez capables de :

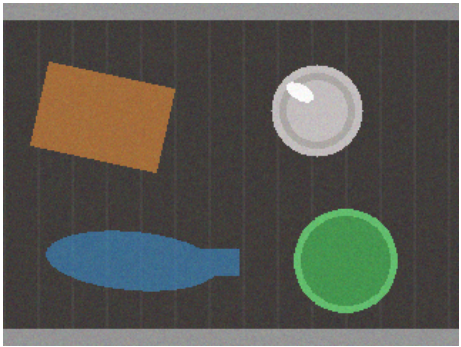
- **Calculer** le coût mémoire et le débit d'une image selon sa résolution et son format, et choisir le format adapté à la tâche.
- **Appliquer** les opérateurs classiques de vision embarquée : seuillage (gris et LAB), morphologie, blobs, contours, Hough, QR codes.
- **Décrire** un objet par des descripteurs géométriques et photométriques (aire, circularité, élongation, couleur moyenne).
- **Expliquer** l'influence de l'éclairage et **mettre en œuvre** une calibration (exposition, gain, balance des blancs).
- **Évaluer** un classifieur à règles par une matrice de confusion et **justifier** le passage à l'apprentissage.

Le fil rouge et le livrable

- La station EcoTri 4.0 doit trier quatre matériaux : **métal, carton, plastique, verre**.
- **Jalon J2** : un classifieur à règles qui tourne sur la Portenta et son **taux d'erreur mesuré**. Il servira de **référence** (*baseline*) pour le réseau de neurones de la lecture 5.

Le problème déclencheur

Une canette, un carton : faut-il vraiment un réseau de neurones ?



Vue de dessus du tapis (scène de synthèse, 320×240) : carton, canette, bouteille plastique, bocal en verre.

L'intuition

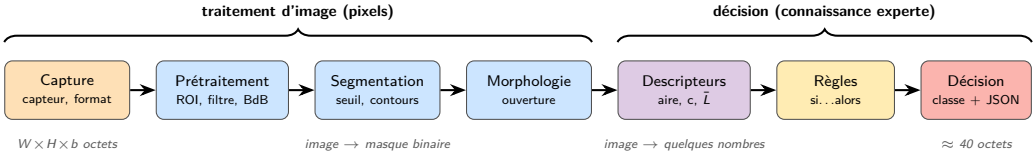
- Le carton est **brun** et **rectangulaire**.
- La canette est **grise**, **ronde** et **brillante**.
- Quelques seuils et un « si . . .alors » devraient suffire.

À vous (5 min, par équipe)

Écrivez **trois règles** qui distinguent les quatre matériaux. Pour chacune, notez **la grandeur mesurée** dans l'image et **un cas** où la règle se trompera.

La chaîne de vision à règles

Des pixels à une décision de quelques octets



- Les premières étapes travaillent sur **tous les pixels** : c'est là que se jouent le temps et la mémoire.
- Les descripteurs **compressent** l'image en quelques nombres.

chaque étape est **conçue à la main** par l'ingénieur (Szeliski, 2022, chap. 3 et 7). En apprentissage profond, les étapes 3 à 6 seront **apprises** à partir d'exemples.

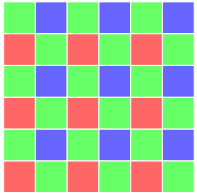
L'image en mémoire

Que coûte une image, et quel format choisir sur un MCU ?

L'image en mémoire

Du photon à l'octet : ce que fait le capteur

- Un capteur CMOS est une matrice de **photosites** : chacun convertit la lumière reçue pendant le temps d'**exposition** en charge, puis en tension.
- Un **gain** analogique amplifie le signal ; un CAN le numérise (8 à 10 bits).
- **Capteur couleur** : un filtre de **Bayer** (R, V, V, B) recouvre les photosites ; la couleur de chaque pixel est **interpolée** (dématriçage).
- **Capteur monochrome** (Vision Shield, HM01B0/HM0360) : pas de filtre, donc plus sensible, mais **aucune information de couleur**.



mosaïque de Bayer RGGB

monochrome

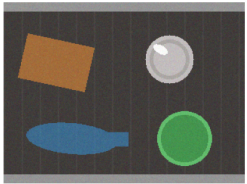
Conséquence pour EcoTri

La Portenta « voit » le monde en **niveaux de gris** : les règles fondées sur la couleur (carton brun, verre vert) devront être portées par la **XIAO** ou l'**ESP32-CAM**, dont les capteurs OV2640 sont en couleur.

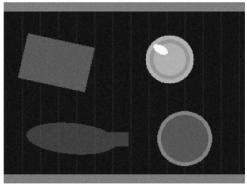
L'image en mémoire

Formats de pixel : ce que l'on garde, ce que l'on perd

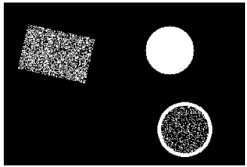
RGB565 : 2 o/pixel (153,6 Ko)



GRAYSCALE : 1 o/pixel (76,8 Ko)



BINAIRE : 1 bit/pixel (9,6 Ko)

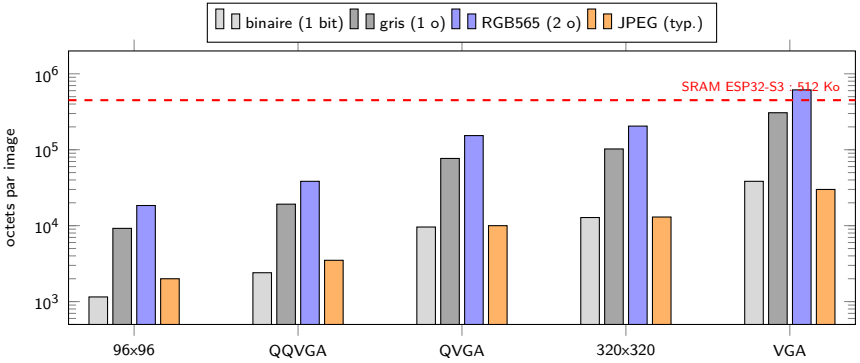


Format	b **(o/pixel)**	**Codage**	**Usage typique**
GRAYSCALE	1	intensité 0–255	formes, contours, QR, entrée de CNN
RGB565	2	R sur 5 bits, V sur 6, B sur 5	couleur, seuils LAB
RGB888	3	8 bits par canal	entrée d'un CNN couleur
YUV422	2	luminance Y + chrominances partagées	sortie native de nombreux capteurs
JPEG	variable	compression avec pertes (DCT)	transmission, stockage
Binaire	1/8	1 bit : objet ou fond	masques après seuillage

L'image en mémoire

Le coût mémoire, résolution par résolution

$$M = W \times H \times b \qquad D = M \times f \times 8$$



Échelle logarithmique : passer de VGA RGB565 à 96×96 en gris divise la mémoire par **67**.

travailler à la **plus petite résolution** et au **format le plus pauvre** qui conservent l'information utile à la décision.

Trois leviers

- **Fenêtrage capteur** (`set_windowing`) : seule une zone est lue ; moins de données transférées.
- **ROI logicielle** (argument `roi=` des fonctions OpenMV) : on ne traite que la bande du tapis sous la caméra.
- **Sous-échantillonnage** (*binning*, `scale`) : moins de pixels, un peu moins de bruit.

Coût d'un opérateur

Un seuillage ou un filtre 3×3 est en $O(WH)$; la transformée de Hough des cercles en $O(N_{\text{contours}} \times N_r)$. Réduire WH d'un facteur 4 réduit le temps d'autant.

```python
import sensor
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
# bande utile du tapis : 320x120
TAPIS = (0, 60, 320, 120)
img = sensor.snapshot()
st = img.get_statistics(roi=TAPIS)
print(st.mean(), st.stdev())
```

! le tapis n'occupe que la moitié centrale de l'image : la ROI divise le temps de traitement par 2 sans rien perdre.

Question flash

?

La XIAO doit envoyer à la Portenta, par BLE (0,7 Mbit/s utiles), une vignette de l'objet pour chaque passage, soit 2 objets par seconde. Quel format et quelle taille de vignette choisir ?

Question flash

? La XIAO doit envoyer à la Portenta, par BLE (0,7 Mbit/s utiles), une vignette de l'objet pour chaque passage, soit 2 objets par seconde. Quel format et quelle taille de vignette choisir ?

Réponse

Budget : 0,7 Mbit/s / 2 = 350 kbit = 43,7 Ko par objet. 96×96 RGB565 = 18,4 Ko (OK, garde la couleur) ; 160×120 RGB565 = 38,4 Ko (limite, sans marge) ; QVGA gris = 76,8 Ko (trop). Choix raisonnable : **96×96 RGB565** ou **JPEG QVGA** ($\approx$ 10 Ko), avec une marge pour les retransmissions.

Segmenter

Comment séparer l'objet du tapis, puis le décrire par quelques nombres ?

Segmenter

Le seuillage : la plus simple des segmentations

- Règle : masque$(x, y) = 1$ si $T_{\text{bas}} \leq I(x, y) \leq T_{\text{haut}}$, sinon 0.
- **Seuil fixe** : choisi à la main (outil *Threshold Editor* d'OpenMV IDE).
- **Seuil d'Otsu** (Otsu, 1979) : le seuil t^* qui maximise la variance inter-classes

$$t^* = \arg\max_t \ \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2$$

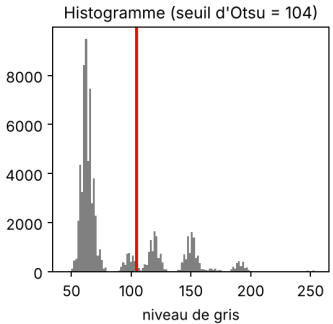
  calculée sur l'histogramme, donc en $O(256)$ une fois l'histogramme connu.
- **Seuil adaptatif** : un seuil par voisinage (utile si l'éclairage n'est pas uniforme).

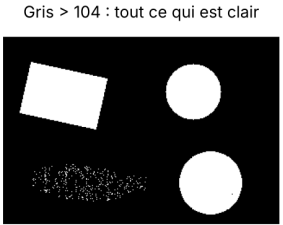
```
img = sensor.snapshot()
h = img.get_histogram(roi=TAPIS)
t = h.get_threshold().value()   # Otsu
img.binary([(t, 255)])          # masque
print("seuil Otsu :", t)
```

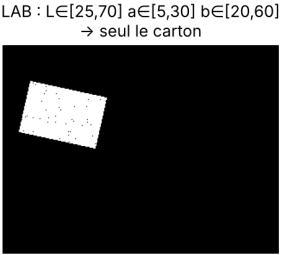
! Otsu suppose un histogramme **bimodal** (un mode « tapis », un mode « objet »). Avec quatre matériaux et des reflets, cette hypothèse tombe.

Segmenter

Seuiller en gris : on sépare le clair du sombre, pas les matériaux



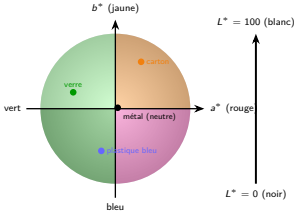




Lecture : à gauche, l'histogramme a plusieurs modes (tapis sombre, objets clairs) ; le seuil d'Otsu (rouge) isole **tout ce qui est clair**, carton, canette et verre confondus. À droite, un seuil dans l'espace **LAB** isole le **seul carton**, grâce à sa chromaticité.

Segmenter

Pourquoi l'espace LAB ?

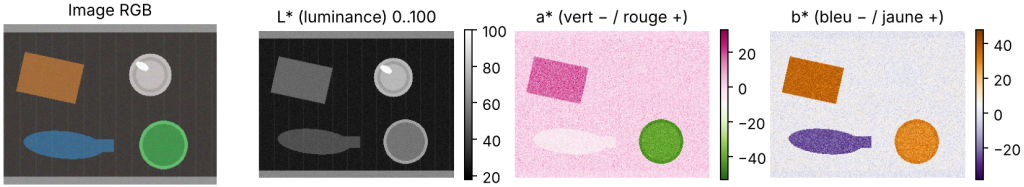


La luminance L^* est séparée de la chromaticité (a^*, b^*) : une variation d'éclairage agit surtout sur L^*. Les distances y sont proches de la perception humaine (CIE 1976).

- En RGB, la luminosité est **mélangée** aux trois canaux : une ombre change R, V et B à la fois.
- CIE L*a*b* (1976) sépare la **luminance** L^* (0 à 100) de la **chromaticité** (a^*, b^*) (environ -128 à $+127$).
- Un seuil couleur devient un **pavé** : $(L_{\min}, L_{\max}, a_{\min}, a_{\max}, b_{\min}, b_{\max})$, le format exact attendu par `find_blobs` dans OpenMV.
- On peut **élargir** la plage de L^* pour tolérer les variations d'éclairage tout en restant strict sur (a^*, b^*).

Segmenter

Les trois canaux LAB de la scène



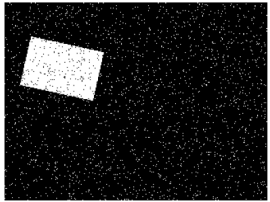
- L^* : canette et carton sont aussi clairs l'un que l'autre.
- a^* : le verre ressort en vert (négatif), le carton légèrement rouge.
- b^* : le carton est très jaune (positif), la bouteille bleue (négatif).

! la canette a $a^* \approx b^* \approx 0$: elle se distingue par sa **forme** (ronde) et ses **reflets**, pas par sa couleur. Il faudra combiner couleur et forme.

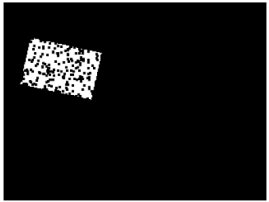
Segmenter

Nettoyer le masque : la morphologie mathématique

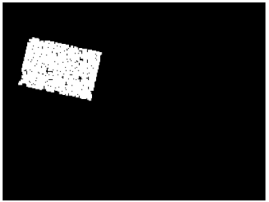
Masque bruité



Érosion 3×3



Ouverture = érosion + dilatation

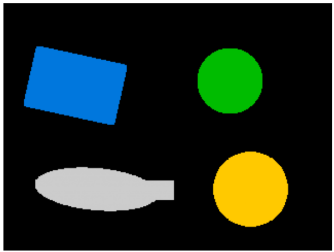


- **Érosion** : un pixel reste à 1 si **tout** son voisinage (élément structurant 3×3) vaut 1 ; supprime les points isolés.
- **Dilatation** : un pixel passe à 1 si **un** voisin vaut 1 ; bouche les trous.
- **Ouverture** = érosion puis dilatation ; **fermeture** = l'inverse (Serra, Mines ParisTech, 1982).

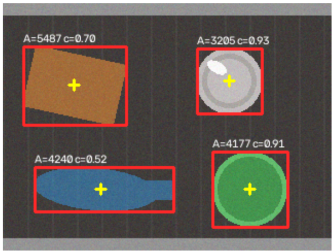
```
img.binary([SEUIL_CARTON])
img.erode(1)    # noyau 3x3
img.dilate(1)   # = ouverture
```

Chaque passage coûte $O(WHk^2)$ avec un noyau $k \times k$: en QVGA et $k = 3$, environ 0,7 M opérations.

Étiquetage en composantes connexes



Blobs : boîte, centroïde, aire A, circularité c



- **Étiquetage** des composantes connexes (connexité 4 ou 8) : chaque objet reçoit un numéro.
- Pour chaque blob : aire A, boîte englobante, centroïde, périmètre P, orientation.

Descripteurs invariants à l'échelle

Circularité $c = 4\pi A/P^2$ (1 pour un disque, $\pi/4 \approx 0{,}79$ pour un carré)
Élongation $= 1 - \lambda_{\min}/\lambda_{\max}$ (moments d'ordre 2)
Densité $= A/A_{\text{boîte}}$ (1 pour un rectangle aligné)

Segmenter

find_blobs dans OpenMV

```
import sensor, time
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)
OBJET = [(110, 255)]          # plus clair que le tapis
clock = time.clock()
while True:
    clock.tick()
    img = sensor.snapshot()
    for b in img.find_blobs(OBJET, roi=(0, 60, 320, 120),
                            pixels_threshold=400, area_threshold=400,
                            merge=True, margin=4):
        img.draw_rectangle(b.rect(), color=255)
        img.draw_cross(b.cx(), b.cy(), color=0)
        print("A=%d  rond=%.2f  elong=%.2f  dens=%.2f"
              % (b.pixels(), b.roundness(), b.elongation(), b.density
      ()))
    print("%.1f FPS" % clock.fps())
```

- `pixels_threshold` et `area_threshold` éliminent le bruit (règle « aire minimale »).
- `merge=True` fusionne les blobs proches (un objet coupé par un reflet).
- En couleur, le seuil devient un pavé LAB à 6 valeurs, par exemple `(25, 70, 5, 30, 20, 60)` pour le carton.
- Le résultat tient en **quelques dizaines d'octets** par objet.

Question flash

?

Une canette vue de dessus a un rayon de 32 pixels. Un carton fait 90×60 pixels. Calculez leur circularité théorique. Une règle « $c > 0{,}8 \Rightarrow$ métal » suffit-elle ? Que se passe-t-il pour un bocal vu de dessus ?

Question flash

?

Une canette vue de dessus a un rayon de 32 pixels. Un carton fait 90×60 pixels. Calculez leur circularité théorique. Une règle « $c > 0{,}8 \Rightarrow$ métal » suffit-elle ? Que se passe-t-il pour un bocal vu de dessus ?

Réponse

Disque : $A = \pi r^2$, $P = 2\pi r$, donc $c = 1$. Rectangle : $A = 5400$, $P = 300$, $c = 4\pi \cdot 5400/90\,000 \approx 0{,}75$. La règle sépare canette et carton, mais le **bocal** est rond lui aussi ($c \approx 1$) : il faut un second critère (couleur, transparence, reflet).

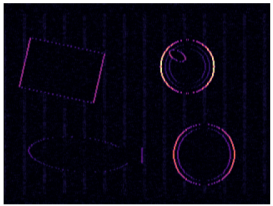
Formes et codes

Comment trouver des contours, des cercles, des droites et lire un QR code ?

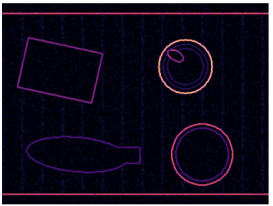
Formes et codes

Contours : là où l'intensité change brusquement

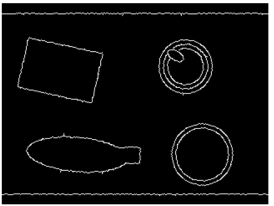
|Gx| (Sobel horizontal)



Norme du gradient



Canny (seuils 60 / 140)



- **Sobel** : deux convolutions 3×3 estiment $G_x = \partial I/\partial x$ et G_y ; la norme $\sqrt{G_x^2 + G_y^2}$ mesure la force du contour.
- **Canny** (MIT, 1986) : lissage gaussien, gradient, **suppression des non-maxima** (contours d'un pixel), **hystérésis** avec deux seuils.

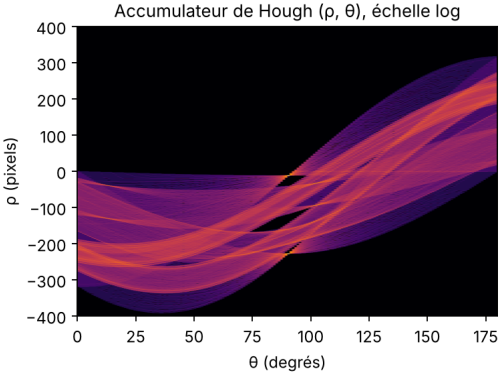
$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

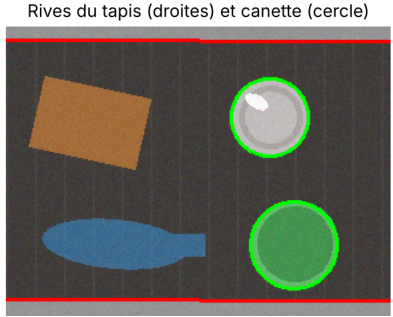
```
img.find_edges(image.EDGE_CANNY,
               threshold=(50, 80))
```

- Une droite s'écrit $\rho = x\cos\theta + y\sin\theta$ (Duda & Hart, 1972).
- Chaque pixel de contour (x, y) **vote** pour toutes les droites qui passent par lui : une sinusoïde dans l'espace (ρ, θ).
- Les **maxima de l'accumulateur** sont les droites présentes, même interrompues ou bruitées.
- **Cercles** : 3 paramètres (a, b, r) ; on borne r (`r_min`, `r_max`) pour limiter le coût.

```
# rives du tapis
for l in img.find_lines(threshold=2500,
                        theta_margin=25, rho_margin=25):
    img.draw_line(l.line(), color=255)
# canettes et bocaux vus de dessus
for c in img.find_circles(threshold=2500,
        r_min=20, r_max=40, r_step=2):
    img.draw_circle(c.x(), c.y(), c.r(), color=255)
```

! l'accumulateur des cercles grandit avec $W \times H \times N_r$: réduire la ROI et la plage de rayons est indispensable sur MCU.





Lecture : à gauche, l'accumulateur (échelle logarithmique) : les points brillants à $\theta \approx 90$ř correspondent aux deux rives horizontales du tapis. À droite, les droites retrouvées (rouge) et les cercles détectés (vert) : la canette **et** le bocal, tous deux ronds.

Formes et codes

QR codes : une forme conçue pour être lue par une machine

QR du lot (version 3)



Vu en perspective : décodé



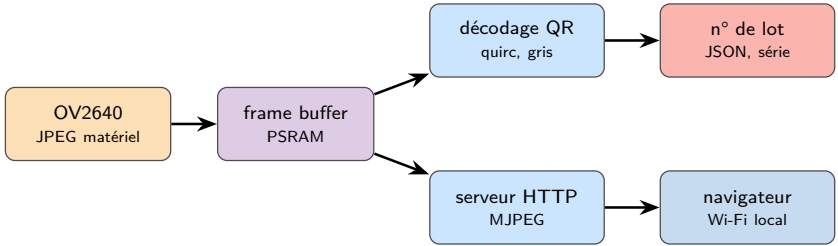
{"lot":"L-2027-0412","site":"FES-01"}

- Norme ISO/IEC 18004 : trois **motifs de repérage** (carrés concentriques au rapport 1 :1 :3 :1 :1) donnent position, orientation et perspective.
- Les modules codent les données avec une **correction d'erreurs** Reed-Solomon (niveaux L, M, Q, H : 7 à 30 % de modules récupérables).
- Un QR version 3 (29×29 modules) contient l'identifiant du lot : `{"lot":"L-2027-0412"}`.
- Dans OpenMV : `img.find_qrcodes()` ; sur ESP32 : bibliothèque **quirc** (via ESP32QRCodeReader).

> **!** pour décoder de façon fiable, viser **au moins 3 à 4 pixels par module**, donc $\geq$ 100 pixels de côté pour un QR version 3.

Formes et codes

La caméra de supervision : QR des lots et flux Wi-Fi



$\approx$ *10 Ko/image QVGA*

- L'OV2640 compresse en **JPEG matériel** : l'ESP32 n'a qu'à envoyer les octets (environ 10 Ko par image QVGA).
- Le décodage QR travaille en **gris** : une seule passe sur 76,8 Ko.

Une seule caméra, deux usages

Décoder et diffuser se disputent le frame buffer et le CPU. Au TP, on mesurera la cadence de chaque usage séparément, puis on discutera de leur partage.

Question flash

?

La caméra de supervision filme une zone de 60 cm de large en QVGA (320 pixels). L'étiquette QR (version 3) d'un lot mesure 4 cm de côté. Sera-t-elle décodée ? Que proposez-vous ?

Question flash

?

La caméra de supervision filme une zone de 60 cm de large en QVGA (320 pixels). L'étiquette QR (version 3) d'un lot mesure 4 cm de côté. Sera-t-elle décodée ? Que proposez-vous ?

Réponse

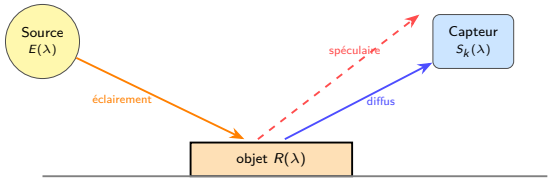
Résolution : $320/60 \approx 5{,}3$ px/cm, donc 4 cm ≈ 21 pixels pour 29 modules : **moins d'un pixel par module**, illisible. Il faut ≥ 3 px/module, soit ≈ 90 px. Solutions : passer en **VGA** ou plus et décoder dans une **ROI** ; **agrandir** l'étiquette à 12 cm ; ou **rapprocher** la caméra de la zone d'entrée des lots.

Lumière et calibration

Pourquoi un seuil réglé le matin ne marche plus l'après-midi ?

Lumière et calibration

Ce que mesure vraiment un pixel



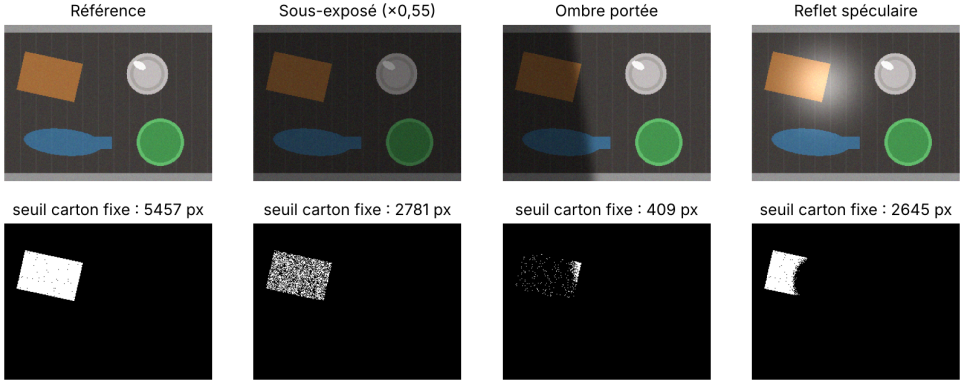
$$I_k = g \int E(\lambda)\, R(\lambda)\, S_k(\lambda)\, d\lambda + \varepsilon$$

la couleur mesurée dépend autant de la lumière E et du gain g que de l'objet R

- La valeur d'un pixel est le produit de la **lumière** E, de la **réflectance** R de l'objet et de la **sensibilité** S_k du capteur, multiplié par le **gain** g et l'exposition.
- La réflexion **diffuse** porte la couleur de l'objet ; la réflexion **spéculaire** (métal, verre, plastique brillant) renvoie la couleur de la **source**.
- Un seuil sur I_k est donc implicitement un seuil sur $E \times g$: il ne vaut que pour **un** éclairage.

Lumière et calibration

Même seuil, quatre éclairages



Lecture : le seuil LAB du carton, réglé sur l'image de référence, perd **la moitié des pixels** en sous-exposition (le bruit l'emporte) et sous un **reflet**, dont la zone saturée sort de la plage. L'ombre qui couvre le carton fait varier L^* mais peu (a^*, b^*).

Lumière et calibration

Les automatismes du capteur : utiles pour l'œil, gênants pour les seuils

- **AE** (exposition auto) et **AGC** (gain auto) maintiennent une luminosité moyenne : quand un carton clair entre dans le champ, **toute l'image s'assombrit**.
- **AWB** (balance des blancs auto) corrige la couleur de la source : la teinte du carton change selon ce qui l'entoure.
- Pour des seuils fixes, il faut **laisser converger**, puis **figer** ces réglages.

```
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)   # convergence
g = sensor.get_gain_db()
e = sensor.get_exposure_us()
sensor.set_auto_gain(False, gain_db=g)
sensor.set_auto_exposure(False, exposure_us=e)
# capteur couleur seulement :
# sensor.set_auto_whitebal(False)
print("gain %.1f dB, expo %d us" % (g, e))
```

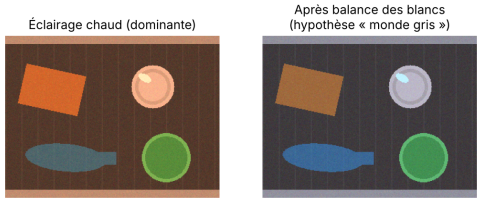
Certaines commandes ne sont pas disponibles sur tous les capteurs : entourez-les d'un `try`.

1. Maîtriser la lumière (le plus efficace)

- **Tunnel d'éclairage** au-dessus du tapis : LED diffuses, aucune lumière du jour.
- Éclairage **rasant** pour les reliefs, **diffus** pour supprimer les reflets du métal.

2. Calibrer la photométrie

- Une **mire grise** (carte 18 %) fixée au bord du tapis sert de référence : on corrige le gain pour que sa moyenne reste constante.
- **Balance des blancs** : hypothèse du « monde gris » (Buchsbaum, 1980), chaque canal est remis à l'échelle de la moyenne.



3. Calibrer la géométrie

- Distorsion de l'objectif : `img.lens_corr(strength=1.8)` ou modèle de Zhang (2000) avec une mire en damier.
- Échelle : mesurer un objet connu donne les **pixels par cm**, pour exprimer les aires en cm^2.

Question flash

?

À 9 h, votre seuil gris `(110, 255)` détecte 98 % des objets. À 15 h, le soleil frappe le tapis et la détection tombe à 60 %. Proposez trois corrections, de la plus robuste à la plus fragile.

Question flash

? À 9 h, votre seuil gris `(110, 255)` détecte 98 % des objets. À 15 h, le soleil frappe le tapis et la détection tombe à 60 %. Proposez trois corrections, de la plus robuste à la plus fragile.

Réponse

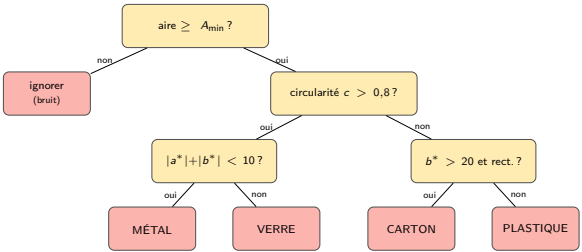
(1) **Supprimer la cause** : tunnel d'éclairage, plus de lumière du jour. (2) **Seuil relatif** : calculer le seuil par Otsu, ou par rapport à la moyenne du tapis vide mesurée en continu sur une ROI de référence. (3) **Recalibrer** : figer gain et exposition, ajuster le gain à partir d'une mire grise. Retoucher le seuil à la main à chaque fois est la plus fragile.

Limites et synthèse

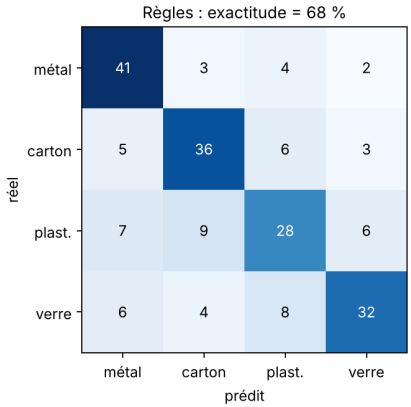
Quel taux d'erreur atteint un classifieur à règles, et pourquoi faudra-t-il apprendre ?

Limites et synthèse

Un classifieur à règles pour EcoTri



- Les règles forment un **arbre de décision écrit à la main**.
- Chaque nœud compare un **descripteur** à un **seuil** choisi par l'ingénieur.
- Avantages : **interprétable**, quelques octets de mémoire, quelques µs de calcul, aucune donnée d'entraînement.
- Sur la Portenta (gris), la branche couleur est remplacée par la **luminance moyenne** et l'**écart-type** (texture, reflets).



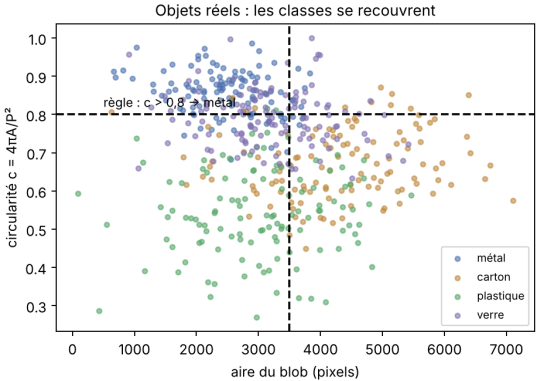
- Ligne = classe **réelle**, colonne = classe **prédite** ; la diagonale compte les bonnes réponses.
- **Exactitude** (*accuracy*) $= \sum_i C_{ii} / \sum_{ij} C_{ij}$; ici $137/200 = 68{,}5$ %.
- Par classe : **précision** $P_k = C_{kk} / \sum_i C_{ik}$, **rappel** $R_k = C_{kk} / \sum_j C_{kj}$, **F1** $= 2PR/(P+R)$.
- Exemple, plastique : $R = 28/50 = 56$ % ; c'est la classe la moins bien reconnue.

Méthode pour le jalon J2

Mesurer sur un **jeu de test** d'objets **non utilisés** pour régler les seuils (au moins 20 par classe), dans les conditions réelles de la station. Sinon, l'erreur est optimiste.

Limites et synthèse

Pourquoi les règles plafonnent : la variabilité des objets réels



- Avec 2 descripteurs, les classes **se recouvrent** : canettes écrasées, cartons pliés, bouteilles froissées, étiquettes colorées, saleté.
- Des seuils **parallèles aux axes** ne peuvent pas suivre des frontières courbes.
- Ajouter des règles pour chaque cas particulier fait exploser l'arbre et le rend **fragile**.

! plutôt que de concevoir les descripteurs et les seuils à la main, on **apprend** les deux à partir d'exemples étiquetés (Goodfellow et al., 2016, chap. 9 ; Janapa Reddi, Harvard).

Limites et synthèse

Règles ou apprentissage : un choix d'ingénieur

Critère	Vision à règles	Apprentissage (CNN, lecture 5)
Données nécessaires	aucune (quelques images pour régler)	centaines à milliers d'images étiquetées
Mémoire	quelques Ko	100 Ko à 1 Mo (INT8)
Calcul	µs à ms	ms à centaines de ms
Robustesse à la variabilité	faible	bonne, si les données la couvrent
Interprétabilité	totale	limitée
Cas d'usage idéal	scène maîtrisée, objets normalisés (QR, pièces usinées)	objets naturels, éclairage variable

! les règles restent idéales pour **détecter** l'arrivée d'un objet, lire les **QR** et **recadrer** la vignette ; le CNN ne **classera** que la vignette. C'est une **cascade** (lecture 6).

Livrables de l'équipe

1. Script OpenMV de **détection et suivi** des objets sur le tapis (blobs, formes), avec un message JSON par objet.
2. **Classifieur à règles** documenté : descripteurs, seuils, arbre.
3. **Matrice de confusion** sur au moins 80 passages (20 par classe), exactitude, rappel par classe.
4. Sketch ESP32-CAM : lecture du **QR du lot** et flux Wi-Fi, avec la cadence mesurée.

Format du message (à garder jusqu'en S7)

```
{"t": 183420, "id": 17,
 "cls": "metal", "conf": null,
 "A": 3205, "c": 0.93,
 "cx": 220, "cy": 75,
 "lot": "L-2027-0412"}
```

! sans le chiffre de J2, on ne pourra pas dire en lecture 5 si le CNN « fait mieux » ni de combien.

Question flash

?

Votre classifieur à règles atteint 88 % d'exactitude sur 80 objets de test, mais le rappel du métal n'est que de 70 % (exigence ENF-2 de J1 : rappel métal $\geq$ 90 %). Analysez la situation et proposez deux pistes, l'une sans apprentissage, l'autre avec.

Question flash

? Votre classifieur à règles atteint 88 % d'exactitude sur 80 objets de test, mais le rappel du métal n'est que de 70 % (exigence ENF-2 de J1 : rappel métal $\geq$ 90 %). Analysez la situation et proposez deux pistes, l'une sans apprentissage, l'autre avec.

Réponse

L'exactitude globale masque une **classe critique** ratée : l'exigence n'est pas tenue. Sans apprentissage : regarder les **faux négatifs** (canettes écrasées ? reflets ?), ajouter un descripteur (taux de pixels saturés, élongation), maîtriser l'éclairage (diffuseur). Avec apprentissage : collecter des images des cas ratés et entraîner un classifieur, en vérifiant le **rappel métal** sur le même jeu de test. Avec 20 canettes, un écart de 70 à 90 % ne représente que 4 objets : augmenter le jeu de test.

À retenir

- **Coût d'une image** : $M = W \times H \times b$; choisir la plus petite résolution et le format le plus pauvre qui gardent l'information utile, et traiter une **ROI**.
- **Segmenter** : seuillage (fixe, Otsu, LAB), morphologie (ouverture, fermeture), composantes connexes.
- **Décrire** : aire, circularité $4\pi A/P^2$, élongation, densité, luminance et couleur moyennes.
- **Formes** : Canny pour les contours, Hough pour les droites et cercles, QR codes pour les identifiants.
- **Lumière** : un seuil ne vaut que pour un éclairage ; maîtriser la lumière, figer AE/AGC/AWB, calibrer avec une mire.
- **Évaluer** : matrice de confusion, exactitude, précision, rappel, sur un jeu de test indépendant.
- **Limite** : les règles plafonnent dès que les objets varient ; elles restent la meilleure **référence** et le meilleur **premier étage** d'une cascade.

Glossaire et références

Terme	Définition courte
ROI	région d'intérêt traitée
Blob	composante connexe d'un masque
LAB	espace couleur CIE L*a*b*
Otsu	seuil automatique par histogramme
Hough	détection de formes par vote
AE / AGC / AWB	exposition, gain, balance des blancs automatiques
Rappel	part des objets d'une classe retrouvés

- R. Szeliski (University of Washington), *Computer Vision : Algorithms and Applications*, 2e éd., Springer, 2022 (chap. 3, 7).
- R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4e éd., Pearson, 2018 (chap. 9 et 10).
- N. Otsu, « A Threshold Selection Method from Gray-Level Histograms », *IEEE Trans. SMC*, 9(1), 1979.
- J. Canny (MIT), « A Computational Approach to Edge Detection », *IEEE PAMI*, 8(6), 1986.
- R. O. Duda, P. E. Hart, « Use of the Hough Transformation to Detect Lines and Curves », *Comm. ACM*, 15(1), 1972.
- J. Serra (Mines ParisTech), *Image Analysis and Mathematical Morphology*, Academic Press, 1982.
- Stanford CS131 et MIT 6.8300 : cours d'introduction à la vision par ordinateur.
- ISO/IEC 18004 :2015, QR Code ; Documentations OpenMV (`image`, `sensor`) et Espressif `esp32-camera`.
- V. Janapa Reddi et al. (Harvard), *Machine Learning Systems*, mlsysbook.ai.

Prochaine étape

Tutorial 2 : vision à règles avec OpenMV sur la Portenta. **TP 2** : détection, suivi et classification sur le tapis, QR et flux sur l'ESP32-CAM, jalon J2.